

Longest Palindromic Subsequence (LPS) - Complete Notes

Problem Statement

Given a string `s`, find the length of the **Longest Palindromic Subsequence (LPS)**.

A subsequence is obtained by deleting some characters (possibly none) without changing the relative order of the remaining characters.

Example

```
s = "bbbab"
Output = 4
LPS = "bbbb"
```

Pattern Identification

Whenever you see:

- Palindrome
- Subsequence
- String interval `[i...j]`
- Choices involving left and right ends of a substring

Think:

Interval DP on a single string.

State generally becomes:

```
f(i,j)
```

meaning:

```
Answer for substring s[i...j]
```

Step 1: Define the State

```
f(i,j)
=
Length of the longest palindromic subsequence in s[i...j]
```

Step 2: Build the Recurrence

The only important characters are:

```
s[i]
s[j]
```

because they are the ends of the current substring.

There are only two cases.

Case 1 : Characters Match

Example:

```
a b c b a
↑       ↑
i       j
```

Both ends are 'a'.

Since a palindrome must have equal ends, we can include both characters.

After taking them:

```
a [b c b] a
```

The remaining problem is:

```
f(i+1,j-1)
```

because we already used $s[i]$ and $s[j]$.

Thus:

```
f(i,j)
=
2 + f(i+1,j-1)
```

Why +2 ?

Because we used:

```
a      a
↑      ↑
```

Two characters are added to the palindrome.

Why (i+1,j-1) ?

Because:

- i has been used.
- j has been used.

The remaining substring is:

```
i+1      j-1
↓        ↓
a b c b a
  b c b
```

Case 2 : Characters Don't Match

Example:

```
a b c b d
↑      ↑
```

$a \neq d$

A palindrome cannot start with **a** and end with **d**.

Therefore, at least one of them must be removed.

Choice 1

Remove left character.

```
f(i+1,j)
```

Choice 2

Remove right character.

```
f(i,j-1)
```

Since we want the **longest** palindrome:

```
f(i,j)
=
max(
    f(i+1,j),
    f(i,j-1)
)
```

Why NOT use **f(i+1,j-1)** ?

Because that removes BOTH characters.

Maybe only one of them needs to be removed.

Removing both may lose the optimal answer.

Therefore:

```
max(
    f(i+1,j),
    f(i,j-1)
)
```

is necessary.

Final Recurrence

```
def f(i,j):  
    if i > j:  
        return 0  
  
    if i == j:  
        return 1  
  
    if s[i] == s[j]:  
        return 2 + f(i+1,j-1)  
  
    return max(  
        f(i+1,j),  
        f(i,j-1)  
    )
```

Base Cases

Empty substring

```
i > j
```

No characters.

Answer:

```
0
```

Single character

```
i == j
```

Every character is a palindrome.

Answer:

1

Recursive Solution

```
def f(i,j):

    if i > j:
        return 0

    if i == j:
        return 1

    if s[i] == s[j]:
        return 2 + f(i+1,j-1)

    return max(
        f(i+1,j),
        f(i,j-1)
    )

def longestPalindromeSubseq(s):
    n = len(s)
    return f(0,n-1)
```

Memoization

```
def longestPalindromeSubseq(s):
    n = len(s)

    dp = [[-1]*n for _ in range(n)]

    def f(i,j):

        if i > j:
            return 0

        if i == j:
            return 1

        if dp[i][j] != -1:
            return dp[i][j]
```

```
    if s[i] == s[j]:
        dp[i][j] = 2 + f(i+1,j-1)
    else:
        dp[i][j] = max(
            f(i+1,j),
            f(i,j-1)
        )

    return dp[i][j]

return f(0,n-1)
```

Complexity

Time:

$O(n^2)$

Space:

$O(n^2)$

TABULATION METHOD (Dependency Order Method)

State

dp[i][j]
=
Length of LPS in s[i...j]

Base Case

dp[i][i] = 1

Recurrence

```
if s[i] == s[j]:
    dp[i][j] = 2 + dp[i+1][j-1]
else:
    dp[i][j] = max(
        dp[i+1][j],
        dp[i][j-1]
    )
```

How to Decide Filling Order

Look at dependencies.

Current cell:

```
dp[i][j]
```

depends on:

```
dp[i+1][j]
dp[i][j-1]
dp[i+1][j-1]
```

These cells are:

- below
- left
- diagonal-left-below

Therefore:

```
i → backward
j → forward
```

Filling Order

```
for i in range(n-1,-1,-1):
    for j in range(i+1,n):
```

Tabulation Code

```
def longestPalindromeSubseq(s):
    n = len(s)

    dp = [[0]*n for _ in range(n)]

    for i in range(n):
        dp[i][i] = 1

    for i in range(n-1,-1,-1):
        for j in range(i+1,n):

            if s[i] == s[j]:
                dp[i][j] = 2 + dp[i+1][j-1]
            else:
                dp[i][j] = max(
                    dp[i+1][j],
                    dp[i][j-1]
                )

    return dp[0][n-1]
```

Complexity

Time:

$O(n^2)$

Space:

$O(n^2)$

Interval DP Recognition Template

If the problem says:

- substring
- interval
- palindrome
- partition
- split at k
- answer for $s[i \dots j]$

Think:

State:
 $f(i, j)$

and consider:

1. Use both ends?
2. Remove left?
3. Remove right?
4. Split at k?

This is the core Interval DP pattern.